

LAB PROGRAM - TOPIC 4

SHAHANAJ C

192372292

**SIMATS Online Programming Lab**

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 4

Language: Python C C++ Java

QUESTIONS

Q1

Closest Pair of Points (Brute Force Approach)

10 marks

Q2

Closest Pair of Points (Brute Force Approach)

10 marks

Q3

Closest Pair of Points (Brute Force Approach)

10 marks

Q4

Closest Pair of Points (Brute Force Approach)

10 marks

Q5

Closest Pair of Points (Brute Force Approach)

10 marks

Q6

Exhaustive Search for Subset Sum

10 marks

Q7

Exhaustive Search for Subset Sum

10 marks

Q8

Exhaustive Search for Subset Sum

10 marks

Q9

Convex Hull (Basic Method)

10 marks

Q10

Q1 Closest Pair of Points (Brute Force Approach)

10 marks

Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.
Input:
◆ A list of points represented by coordinates (x, y).
Points: [(1, 2), (4, 5), (7, 8), (3, 1)]
Output:
◆ The two points with the minimum distance.
◆ The minimum distance between them.
Example Output:
Closest pair: (1, 2) - (3, 1)
Minimum distance: 1.4142135623730951

Your Code

```
1 import math
2
3 def find_closest_pair(points):
4     if len(points) < 2:
5         return None, float('inf')
6
7     min_dist = float('inf')
8     closest_pair = None
9     for i in range(len(points)):
10        for j in range(i + 1, len(points)):
11            p1 = points[i]
12            p2 = points[j]
13            dist = math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
14
15            if dist < min_dist:
16                min_dist = dist
17                closest_pair = (p1, p2)
18
19     return closest_pair, min_dist
20
21 if __name__ == "__main__":
22
23     points = [(1, 2), (4, 5), (7, 8), (3, 1)]
24
25     pair, distance = find_closest_pair(points)
```

Input

stdin input

Output

Closest pair: (1, 2) - (3, 1)
Minimum distance: 1.4142135623730951

**SIMATS Online Programming Lab**

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 4

Language: Python C C++ Java

QUESTIONS

Q1

Closest Pair of Points (Brute Force Approach)

10 marks

Q2

Closest Pair of Points (Brute Force Approach)

10 marks

Q3

Closest Pair of Points (Brute Force Approach)

10 marks

Q4

Closest Pair of Points (Brute Force Approach)

10 marks

Q5

Closest Pair of Points (Brute Force Approach)

10 marks

Q6

Exhaustive Search for Subset Sum

10 marks

Q7

Exhaustive Search for Subset Sum

10 marks

Q8

Exhaustive Search for Subset Sum

10 marks

Q9

Convex Hull (Basic Method)

10 marks

Q10

Q2 Closest Pair of Points (Brute Force Approach)

10 marks

Develop a program to calculate the shortest distance between any two points using the brute force closest pair algorithm.
Input:
Points: [(0,0), (2,2), (3,1), (6,5)]
Output:
Closest pair (2,2) - (3,1)
Minimum distance: 1.4142135623730951

Your Code

```
1 import math
2
3 def calculate_distance(p1, p2):
4     """Calculates the Euclidean distance between two points."""
5     return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
6
7 def closest_pair_brute_force(points):
8     """Finds the closest pair of points using the brute force approach.
9     if n < 2:
10        return None, float('inf')
11
12     min_dist = float('inf')
13     closest_pair = (None, None)
14     for i in range(n):
15        for j in range(i + 1, n):
16            dist = calculate_distance(points[i], points[j])
17            if dist < min_dist:
18                min_dist = dist
19                closest_pair = (points[i], points[j])
20
21     return closest_pair, min_dist
22
23
24 def main():
25
26     points = [(0,0), (2,2), (3,1), (6,5)]
27
28     pair, distance = closest_pair_brute_force(points)
29
30     if pair[0] is not None:
```

Input

stdin input

Output

Closest pair (2, 2) - (3, 1)
Minimum distance:
1.4142135623730951



SIMATS
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

 CHERUVU SHAHANAJ
19237229RA

Back

LAB PROGRAM -TOPIC 4

Language: Python C C++ Java

Q1
Closest Pair of Points (Brute Force Approach)
10 marks

Q2
Closest Pair of Points (Brute Force Approach)
10 marks

Q3
Closest Pair of Points (Brute Force Approach)
10 marks

Q4
Closest Pair of Points (Brute Force Approach)
10 marks

Q5
Closest Pair of Points (Brute Force Approach)
10 marks

Q6
Exhaustive Search for Subset Sum
10 marks

Q7
Exhaustive Search for Subset Sum
10 marks

Q8
Exhaustive Search for Subset Sum
10 marks

Q9
Convex Hull (Basic Method)
10 marks

Q10
Convex Hull (Basic Method)
10 marks

Q3 Closest Pair of Points (Brute Force Approach)

10 marks

Write a program to identify the closest pair of points from a given collection of 2D points using brute force technique.

Input:
Points: [[10,20], (15,25), (30,40), (12,22)]

Output:
Closest pair: (10,20) - (12,22)
Minimum distance: 2.8284271247461903

Your Code

```
1 import math
2
3 def closest_pair(points):
4     n = len(points)
5     min_dist = float('inf')
6     closest = None
7
8     for i in range(n):
9         for j in range(i + 1, n):
10            x1, y1 = points[i]
11            x2, y2 = points[j]
12            dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
13
14            if dist < min_dist:
15                min_dist = dist
16                closest = (points[i], points[j])
17
18    return closest, min_dist
19 points = [(10, 20), (15, 25), (30, 40), (12, 22)]
20
21 pair, min_distance = closest_pair(points)
22
23 print(f"Closest pair: {(pair[0][0]}, {(pair[0][1])}) - {(pair[1][0]}, {(pair[1][1])})")
24 print(f"Minimum distance: {min_distance}")
```

Submitted

Input

stdin input

Output

```
Closest pair: (10,20) - (12,22)
Minimum distance:
2.8284271247461903
```



SIMATS
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

 CHERUVU SHAHANAJ
19237229RA

Back

LAB PROGRAM -TOPIC 4

Language: Python C C++ Java

Q1
Closest Pair of Points (Brute Force Approach)
10 marks

Q2
Closest Pair of Points (Brute Force Approach)
10 marks

Q3
Closest Pair of Points (Brute Force Approach)
10 marks

Q4
Closest Pair of Points (Brute Force Approach)
10 marks

Q5
Closest Pair of Points (Brute Force Approach)
10 marks

Q6
Exhaustive Search for Subset Sum
10 marks

Q7
Exhaustive Search for Subset Sum
10 marks

Q8
Exhaustive Search for Subset Sum
10 marks

Q9
Convex Hull (Basic Method)
10 marks

Q10
Convex Hull (Basic Method)
10 marks

Q4 Closest Pair of Points (Brute Force Approach)

10 marks

Write a program to find the closest pair of points among the given set of coordinates using exhaustive comparison of all pairs.

Input:
Points: [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]

Output:
Closest pair: (2, 3) - (5, 4)
Minimum distance: 3.1622776601683795

Your Code

```
1 import math
2
3 points = [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]
4
5 min_dist = float('inf')
6 closest_pair = None
7
8 n = len(points)
9 for i in range(n):
10    for j in range(i + 1, n):
11        x1, y1 = points[i]
12        x2, y2 = points[j]
13        dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
14
15        if dist < min_dist:
16            min_dist = dist
17            closest_pair = (points[i], points[j])
18
19 print("Closest pair:", closest_pair[0], "-", closest_pair[1])
20 print("Minimum distance:", min_dist)
```

Submitted

Input

stdin input

Output

```
Closest pair: (5, 4) - (4, 6)
Minimum distance: 2.23606797749979
```

LAB PROGRAM -TOPIC 4

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Closest Pair of Points (Brute Force Approach)
10 marks

Q2

Closest Pair of Points (Brute Force Approach)
10 marks

Q3

Closest Pair of Points (Brute Force Approach)
10 marks

Q4

Closest Pair of Points (Brute Force Approach)
10 marks

Q5

Closest Pair of Points (Brute Force Approach)
10 marks

Q6

Exhaustive Search for Subset Sum
10 marks

Q7

Exhaustive Search for Subset Sum
10 marks

Q8

Exhaustive Search for Subset Sum
10 marks

Q9

Convex Hull (Basic Method)
10 marks

Q5 Closest Pair of Points (Brute Force Approach)

10 marks

Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7(6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the same line?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

Your Code

```
1 import math
2
3 def calculate_distance(p1, p2):
4     """Calculates the Euclidean distance between two points."""
5     return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
6
7 def find_closest_pair(points):
8     """Finds the closest pair of points using the brute force approach.
9     n = len(points)
10    if n < 2:
11        return None, float('inf')
12
13    min_dist = float('inf')
14    closest_pair = None
15    for i in range(n):
16        for j in range(i + 1, n):
17            dist = calculate_distance(points[i], points[j])
18            if dist < min_dist:
19                min_dist = dist
20                closest_pair = (points[i], points[j])
21
22    return closest_pair, min_dist
23 if __name__ == "__main__":
24
25    sample_points = [(1, 2), (4, 6), (1, 3), (10, 10), (6, 1)]
26
```

Input

stdin input

Output

```
The closest pair is: ((1, 2), (1, 3))
The minimum distance is: 1.0000
```

LAB PROGRAM -TOPIC 4

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Closest Pair of Points (Brute Force Approach)
10 marks

Q2

Closest Pair of Points (Brute Force Approach)
10 marks

Q3

Closest Pair of Points (Brute Force Approach)
10 marks

Q4

Closest Pair of Points (Brute Force Approach)
10 marks

Q5

Closest Pair of Points (Brute Force Approach)
10 marks

Q6

Exhaustive Search for Subset Sum
10 marks

Q7

Exhaustive Search for Subset Sum
10 marks

Q8

Exhaustive Search for Subset Sum
10 marks

Q9

Convex Hull (Basic Method)
10 marks

Q6 Exhaustive Search for Subset Sum

10 marks

Write a program to find whether a subset exists with a given sum using exhaustive search technique.

Input:

Set: [3, 34, 4, 12, 5, 2]

Target Sum: 9

Output:

Subset found: [4, 5]

Sum: 9

Your Code

```
1 from itertools import combinations
2
3 def subset_sum_exhaustive(arr, target):
4     n = len(arr)
5     for r in range(1, n + 1):
6         for subset in combinations(arr, r):
7             if sum(subset) == target:
8                 return subset
9
10    return None
11
12 arr = [3, 34, 4, 12, 5, 2]
13 target = 9
14
15 result = subset_sum_exhaustive(arr, target)
16
17 if result is not None:
18     print("Subset found:", list(result))
19     print("Sum:", target)
20 else:
21     print("No subset found.")
```

Submitted

Input

stdin input

Output

```
Subset found: [4, 5]
Sum: 9
```

LAB PROGRAM - TOPIC 4

← Back

Language: Python C C++ Java

QUESTIONS

Q1 ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q2** ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q3** ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q4** ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q5** ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q6** ✓
Exhaustive Search for Subset Sum
10 marks**Q7** ✓
Exhaustive Search for Subset Sum
10 marks**Q8** ✓
Exhaustive Search for Subset Sum
10 marks**Q9** ✓
Convex Hull (Basic Method)
10 marks**Q10** ✓

10/10 answered

Q7 Exhaustive Search for Subset Sum

10 marks

Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

1. Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])
2. More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

Your Code

```
1 import itertools
2 import math
3
4 def calculate_distance(p1, p2):
5     return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
6
7 def solve_tsp_exhaustive(cities):
8     num_cities = len(cities)
9     if num_cities == 0:
10         return 0, []
11     start_city = cities[0]
12     other_cities = cities[1:]
13
14     min_distance = float('inf')
15     best_path = []
16     for permutation in itertools.permutations(other_cities):
17         current_path = [start_city] + list(permutation) + [start_city]
18         current_distance = 0
19
20         for i in range(len(current_path) - 1):
21             current_distance += calculate_distance(current_path[i], current_path[i+1])
22
23     return min_distance, best_path
```

Input

stdin input

Output

```
Test Case 1:
Shortest Distance:
16.969112047670894
Shortest Path: [(1, 2), (7, 1),
(4, 5), (3, 6), (1, 2)]
Test Case 2:
```

Q2 ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q3** ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q4** ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q5** ✓
Closest Pair of Points (Brute Force Approach)
10 marks**Q6** ✓
Exhaustive Search for Subset Sum
10 marks**Q7** ✓
Exhaustive Search for Subset Sum
10 marks**Q8** ✓
Exhaustive Search for Subset Sum
10 marks**Q9** ✓
Convex Hull (Basic Method)
10 marks**Q10** ✓
Convex Hull (Basic Method)
10 marks

cost by summing the corresponding costs from the cost matrix implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

Test Cases:

Input

1. Simple Case: Cost Matrix: [[3, 10, 7], [8, 5, 12], [4, 6, 9]]
2. More Complex Case: Cost Matrix: [[15, 9, 4], [8, 7, 18], [6, 12, 11]]

Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24

Your Code

```
1 import itertools
2
3 def total_cost(assignment, cost_matrix):
4
5     current_total = 0
6     for worker_idx, task_idx in enumerate(assignment):
7         current_total += cost_matrix[worker_idx][task_idx]
8     return current_total
9
10 def assignment_problem(cost_matrix):
11
12     num_workers = len(cost_matrix)
13     num_tasks = len(cost_matrix[0])
14
15     task_indices = list(range(num_tasks))
16
17     min_cost = float('inf')
18     best_assignment_indices = None
19
20     for p in itertools.permutations(task_indices):
21
22         current_cost = total_cost(p, cost_matrix)
23
24         if current_cost < min_cost:
25             min_cost = current_cost
26             best_assignment_indices = p
27
28     formatted_assignment = []
29     for i, task_idx in enumerate(best_assignment_indices):
30
31         formatted_assignment.append((i, task_idx))
```

Input

stdin input

Output

```
Test Case 1:
Optimal Assignment: [(worker 1,
task 2), (worker 2, task 2),
(worker 3, task 1)]
Total Cost: 16
```

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

 Language: Python C C++ Java
QUESTIONS

Q1 Closest Pair of Points (Brute Force Approach) 10 marks

Q2 Closest Pair of Points (Brute Force Approach) 10 marks

Q3 Closest Pair of Points (Brute Force Approach) 10 marks

Q4 Closest Pair of Points (Brute Force Approach) 10 marks

Q5 Closest Pair of Points (Brute Force Approach) 10 marks

Q6 Exhaustive Search for Subset Sum 10 marks

Q7 Exhaustive Search for Subset Sum 10 marks

Q8 Exhaustive Search for Subset Sum 10 marks

Q9 Convex Hull (Basic Method) 10 marks

Q10 Convex Hull (Basic Method) 10 marks

Q9 Convex Hull (Basic Method) 10 marks

Develop a program to find the outer boundary of a set of points using the convex hull algorithm.

Input:
Points: [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]

Output:
Convex Hull Points:
(4,1), (6,3), (5,5), (3,6), (1,4), (2,2)

Your Code

```

1 def cross(o, a, b):
2
3     return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
4
5 def convex_hull(points):
6
7     points = sorted(set(points))
8     if len(points) <= 1:
9         return points
10
11     lower = []
12     for p in points:
13         while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
14             lower.pop()
15             lower.append(p)
16
17     upper = []
18     for p in reversed(points):
19         while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
20             upper.pop()
21             upper.append(p)
22
23     return lower[:-1] + upper[::-1]
24
25 pts = [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]
26
27 hull = convex_hull(pts)
28 print("Convex Hull Points:")
29 for p in hull:
30     print(p)
    
```

Input

stdin input

Output

Convex Hull Points:
(4, 1)
(6, 3)
(5, 5)
(3, 6)
(1, 4)
(2, 2)

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

 Language: Python C C++ Java
QUESTIONS

Q1 Closest Pair of Points (Brute Force Approach) 10 marks

Q2 Closest Pair of Points (Brute Force Approach) 10 marks

Q3 Closest Pair of Points (Brute Force Approach) 10 marks

Q4 Closest Pair of Points (Brute Force Approach) 10 marks

Q5 Closest Pair of Points (Brute Force Approach) 10 marks

Q6 Exhaustive Search for Subset Sum 10 marks

Q7 Exhaustive Search for Subset Sum 10 marks

Q8 Exhaustive Search for Subset Sum 10 marks

Q9 Convex Hull (Basic Method) 10 marks

Q10 Convex Hull (Basic Method) 10 marks

Q10 Convex Hull (Basic Method) 10 marks

Write a program to calculate the convex hull of given points and display the points forming the polygon boundary.

Input:
Points: [(1,1), (2,4), (5,5), (6,2), (3,0)]

Output:
Convex Hull Points:
(3,0), (6,2), (5,5), (2,4)

Your Code

```

1 def convex_hull(points):
2     points.sort()
3     if len(points) <= 2:
4         return points
5     def cross_product(o, a, b):
6         return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
7
8     lower = []
9     for p in points:
10        while len(lower) >= 2 and cross_product(lower[-2], lower[-1], p) <= 0:
11            lower.pop()
12            lower.append(p)
13
14    upper = []
15    for p in reversed(points):
16        while len(upper) >= 2 and cross_product(upper[-2], upper[-1], p) <= 0:
17            upper.pop()
18            upper.append(p)
19    return lower[:-1] + upper[::-1]
20
21 points = [(1, 1), (2, 4), (5, 5), (6, 2), (3, 0)]
22 hull = convex_hull(points)
23 print("Convex Hull Points:")
24 print(" ".join(map(str, hull)))
    
```

Input

stdin input

Output

Convex Hull Points:
(1, 1), (3, 0), (6, 2), (5, 5), (2, 4)

Submitted